

CKAN: Collaborative Knowledge-aware Attentive Network for Recommender Systems

Ze Wang
Beihang University
wangzzz@buaa.edu.cn

Guangyan Lin
Beihang University
lingy@buaa.edu.cn

Huobin Tan*
Beihang University
thbin@buaa.edu.cn

Qinghong Chen
Beihang University
chen.qinghong@buaa.edu.cn

Xiyang Liu
Beihang University
liuxiyang@buaa.edu.cn

ABSTRACT

Since it can effectively address the problem of sparsity and cold start of collaborative filtering, *knowledge graph* (KG) is widely studied and employed as side information in the field of recommender systems. However, most of existing KG-based recommendation methods mainly focus on how to effectively encode the **knowledge associations** in KG, without highlighting the crucial **collaborative signals** which are latent in user-item interactions. As such, the learned embeddings underutilize the two kinds of pivotal information and are insufficient to effectively represent the latent semantics of users and items in vector space.

In this paper, we propose a novel method named *Collaborative Knowledge-aware Attentive Network* (CKAN) which explicitly encodes the collaborative signals by collaboration propagation and proposes a natural way of combining collaborative signals with knowledge associations together. Specifically, CKAN employs a heterogeneous propagation strategy to explicitly encode both kinds of information, and applies a knowledge-aware attention mechanism to discriminate the contribution of different knowledge-based neighbors. Compared with other KG-based methods, CKAN provides a brand-new idea of combining collaborative information with knowledge information together. We apply the proposed model on four real-world datasets, and the empirical results demonstrate that CKAN significantly outperforms several compelling state-of-the-art baselines.

CCS CONCEPTS

• Information systems → Recommender systems.

KEYWORDS

Recommender Systems; Heterogeneous Propagation; Knowledge-aware Attention Mechanism; Knowledge Graph

*Huobin Tan is the corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](https://www.acm.org/permissions).

SIGIR '20, July 25–30, 2020, Virtual Event, China

© 2020 Association for Computing Machinery.

ACM ISBN 978-1-4503-8016-4/20/07...\$15.00

<https://doi.org/10.1145/3397271.3401141>

ACM Reference Format:

Ze Wang, Guangyan Lin, Huobin Tan, Qinghong Chen, and Xiyang Liu. 2020. CKAN: Collaborative Knowledge-aware Attentive Network for Recommender Systems. In *43rd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '20)*, July 25–30, 2020, Virtual Event, China. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3397271.3401141>

1 INTRODUCTION

Ranging from E-commerce, social platforms to news websites, recommender system has become an integral part in almost every application that provides information service. Among recommendation strategies, the classical *collaborative filtering* (CF) [6, 10, 29, 34], which assumes that users with similar behaviors may have similar appetites for items, has been widely studied and employed. However, these CF-based methods usually suffer from sparsity and cold-start problem in default of side information.

Among various kinds of side information, *knowledge graph* (KG), which contains much more additional facts and connections about items, has gained growing attention [1, 19, 23, 26, 27]. The core in the fusion of knowledge graph and recommender systems is how to effectively integrate such heterogeneous information into latent vector representations of users and items. According to how to deal with this challenge, existing KG-based recommender systems can be roughly categorized into three types, embedding-based [1, 21, 22, 33], path-based [7, 15, 16, 27, 31, 32] and propagation-based [14, 17, 20, 23, 24, 26]:

- In embedding-based methods, the entity embeddings that are pre-trained with *knowledge graph embedding* (KGE) [25, 35] in KG serve as item representations in recommendation. For example, CKE [33] learns item embeddings with the involvement of KG via TransR [11]. DKN [21] regards entity embeddings and word embeddings as different channels and generates news embeddings for recommendation by utilizing KG via TransD[8]. KTUP [1] adopts TransH [28] to learn a translation-based model which has a excellent performance on both tasks of recommendation and KG completion. However, since the KGE methods adopted in these models are more applicable for in-graph applications than recommendation, the learnt entity embeddings are insufficient for item recommendations.
- As for path-based algorithms, multiple patterns of connections among entities in KG are proposed to provide extra guidance for recommendation. There are two ways to cope with the great many paths between entities. One is to apply selection algorithm

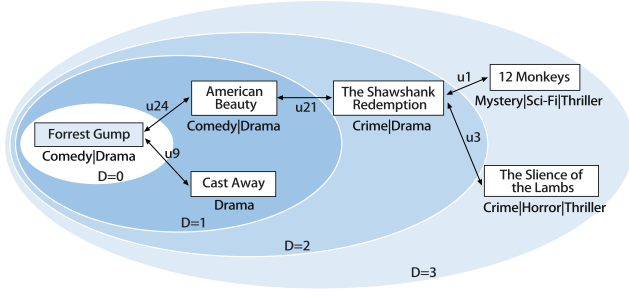


Figure 1: An example of noise introduced by high-order interactions in MovieLens-20M. The links (e.g. u24) mean user with id 24 who interacted with movies on both ends. The genres of Forrest Gump is completely different from its high-order interactive neighbors’.

to select prominent paths [16, 27]. The other is to define meta-path patterns to constrain the paths [7, 31]. The primary problem of path-based methods is that it is hard to optimize path selection in practice. In addition, domain knowledge is required to define effective meta-paths, which is super time-consuming and labor-intensive to design meta-paths manually, especially in an extremely complicated KG.

- Propagation-based methods, where propagation is performed iteratively over the whole KG to find auxiliary information for recommendation, have attracted much attention recently. RippleNet [20], which makes great progress in KG-based recommendation, explores user’s potential interests along links in KG, but the significance of connectivity is weakly characterized in the framework. KGCN [24] and KGNN-LS [23] adopt *Graph convolutional network* (GCN) [30] to obtain item embeddings via their neighbors in KG, which show neighbors’ information is capable of leading to a big boost in the recommendation task. But note that the explicit collaborative signals are ignored in both two methods, which results in insufficient item embeddings. KGAT [26] proposes *collaborative knowledge graph* (CKG) to combine *user-item bipartite graph* (UIG) with *knowledge graph* (KG) together and recursively performs propagation over CKG via GCN to complement entity embeddings. One deficiency in KGAT is that it regards users as the same nodes as entities in CKG, which makes it difficult to deal with new users. Once there is a new interacted user, KGAT needs to reconstruct the CKG and retrain the whole model to make recommendations for him, the computing costs of which are unacceptable in industry. Moreover, KGAT assumes that the interacted items in UIG and the associated entities in KG should be viewed as the homogeneous nodes, but in fact, they are in different latent spaces. Furthermore, because the types of items that users have interacted with are usually very broad, the high-order interactions in UIG might have completely different meanings with the latent semantic representation of original item, which would introduce noise into embeddings. For example, as is depicted in Figure 1, the film *Forrest Gump*, labeled as *comedy|drama*, propagates to the

completely unrelated film *12 Monkeys* that is categorized as *mystery|sci-fi|thriller* via three-order user-item interaction propagation.

To address the limitation of existing methods of KG-based recommendation, we propose an end-to-end model named CKAN, short for *Collaborative Knowledge-aware Attentive Network*, which seamlessly combines explicitly encoded collaborative signals in user-item interactions and auxiliary knowledge associations in KG. Specifically, CKAN is equipped with two designs: 1) Heterogeneous propagation, which is composed of collaboration propagation and knowledge graph propagation, views interaction and knowledge as information in two different spaces and combines them in a natural way, so that they share different weights to contribute to the embeddings. 2) Knowledge-aware attentive embedding, which is a brand new knowledge-aware neural attention mechanism proposed for learning different weights of neighbors in KG when they are in different conditions.

Empirically, We apply CKAN in four real-world scenarios of music, book, movie and restaurant recommendation. The experimental results show that our proposed method CKAN distinctly outperforms state-of-the-art algorithms especially in click-through rate prediction, with *AUC* increased by 1.6%, 2.9%, 1.7% over state-of-the-art baselines in music, book and restaurant datasets, respectively. The code and data are publicly accessible at <https://github.com/weberrr/CKAN> for researchers to validate and conduct further researches.

To sum up, our contributions are as follows:

- We propose a new recommendation method called CKAN, an end-to-end framework that effectively combines explicitly encoded collaborative signals latent in interactions with auxiliary knowledge associations in KG together.
- We perform empirical experiments on four real-world recommendation scenarios whose results demonstrate the superiority of CKAN over compelling state-of-the-art baselines.

2 PROBLEM FORMULATION

Following the formulation of mainstream recommender models, we assume that there is a set of M users $\mathcal{U} = \{u_1, u_2, \dots, u_M\}$ and a set of N items $\mathcal{V} = \{v_1, v_2, \dots, v_N\}$ in a certain recommendation scenario. According to users’ implicit feedback in the log history, such as clicking, watching and purchasing, we can get a user-item interaction matrix $\mathbf{Y} \in \mathbb{R}^{M \times N}$, where $y_{uv} = 1$ indicates that user u has interacted with item v , otherwise $y_{uv} = 0$. Note that the negative implicit feedback ($y_{uv} = 0$) doesn’t mean that the user u dislikes the item v , maybe user u likes item v but just ignores or skips it by accident. In addition, we organize the side information in the form of knowledge graph $\mathcal{G} = \{(h, r, t) | h, t \in \mathcal{E}, r \in \mathcal{R}\}$ in which each knowledge triple (h, r, t) denotes that there is a relationship r between head entity h and tail entity t , $\mathcal{E}$ and $\mathcal{R}$ are the sets of entities and relations in the knowledge graph. For example, the triple $(Leonardo\ DiCaprio, ActorOf, The\ Great\ Gatsby)$ states the fact that *Leonardo* is an actor of the movie *The Great Gatsby*. Note that, $\mathcal{R}$ contains relations in both canonical direction (i.e. *ActorOf*) and inverse direction (i.e. *ActedBy*). We employ a set $\mathcal{A} = \{(v, e) | v \in \mathcal{V}, e \in \mathcal{E}\}$ where (v, e) indicates that item v can

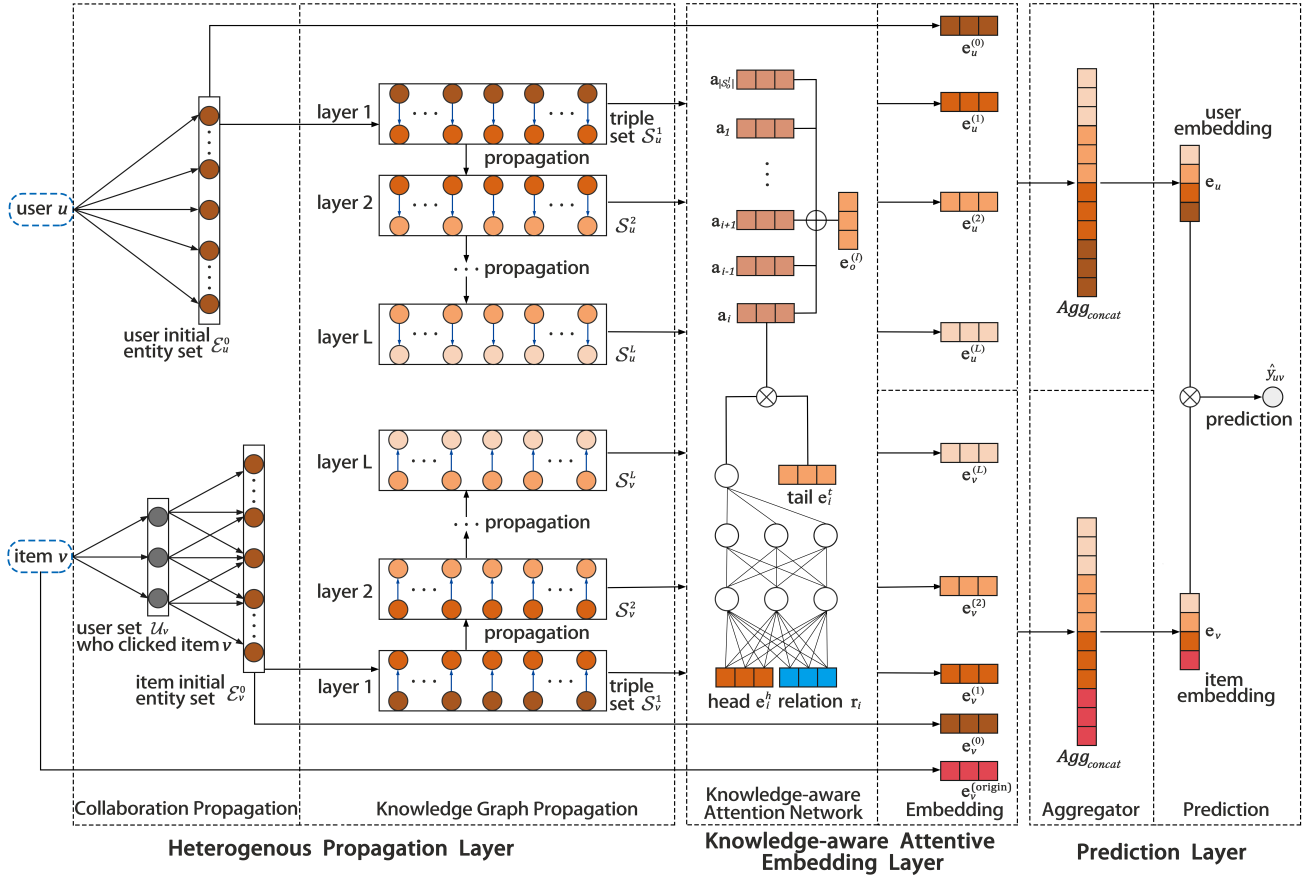


Figure 2: Illustration of the proposed CKAN model, which consists of three modules: heterogeneous propagation layer, knowledge-aware attentive embedding layer and prediction layer. Best view in color.

be aligned with entity e in the knowledge graph to elucidate alignments between items and entities.

Since we get user-item interaction matrix $\mathbf{Y}$ and knowledge graph $\mathcal{G}$, now the task is to predict what is the probability that the user u will have interaction with items which he has not engaged with before. Specifically, our paramount goal is to learn a prediction function $\hat{y}_{uv} = \mathcal{F}(u, v | \Theta, \mathcal{G})$, where $\hat{y}_{uv}$ indicates the predicted probability and Θ denotes the model parameters of function $\mathcal{F}$.

3 METHODOLOGY

We now detail the framework of the proposed CKAN model. As shown in Figure 2, the model framework consists of three main components: 1) heterogeneous propagation layer (heterogeneity refers to two different propagation structures) that propagates the collaborative signals through user-item interactions and knowledge associations in knowledge graph; 2) knowledge-aware attentive embedding layer, which presents a new neural knowledge-aware attention mechanism to learn the knowledge-based weights

of entities in the same entity set and generates weighted representation of entities; and 3) prediction layer, that aggregates the representations of both user and item from different propagation layers and outputs the predicted click probability.

3.1 Heterogeneous Propagation

As shown in Figure 2, the heterogeneous propagation layer is composed of two major modules: **collaboration propagation** and **knowledge graph propagation**. The collaboration propagation is used to explicitly encode crucial collaborative signals into representations of both user and item. The knowledge graph propagation propagates knowledge associations along links in KG to expand representations of both user and item with supplementary side information recursively. Next we will discuss each part exhaustively.

3.1.1 Collaboration Propagation. Intuitively, items that a user has interacted with in history are capable of representing the user's preferences to a certain extent. Rather than traditionally using an independent latent vector, we represent user u by his related items.

The related item set of user u , obtained through his historical interactions, can be converted into the initial seed set propagated in KG through the alignments between items and entities. The initial entity set of user u is defined as follows:

$$\mathcal{E}_u^0 = \{e | (v, e) \in \mathcal{A} \text{ and } v \in \{v | y_{uv} = 1\}\} \quad (1)$$

Analogously, users who have interacted with the same item can also contribute to the item's feature representation as a result of their similar behavior preferences. We define items that have been interacted by the same user as collaborative neighbors and formulate the collaborative item set of item v as follows:

$$\mathcal{V}_v = \{v_u | u \in \{u | y_{uv} = 1\} \text{ and } y_{uv_u} = 1\} \quad (2)$$

Synthesizing collaborative item set and alignment set, the initial entity set of item v is defined as follows:

$$\mathcal{E}_v^0 = \{e | (v_u, e) \in \mathcal{A} \text{ and } v_u \in \mathcal{V}_v\} \quad (3)$$

The collaboration propagation layer is able to explicitly encode the first-order interactive information that most effectively expresses the latent semantics into the initial entity set, thereby enhancing the representations of users and items and improving the recommendation effect. Notice that, by definition, the initial entity set of item v also contains the entity directly associated with item v . Such a design can continuously emphasize information of original item v and reduce bias due to multi-layer propagation.

3.1.2 Knowledge Graph Propagation. Neighboring entities in KG always have strong associations. Propagating along links in KG, we can obtain extended entity sets and triple sets of different distances from the initial entity set, which are capable of extending the latent vector representations of user and item effectively. The definition of entity set for user u and item v is recursively formulated as:

$$\mathcal{E}_o^l = \{t | (h, r, t) \in \mathcal{G} \text{ and } h \in \mathcal{E}_o^{l-1}\}, \quad l = 1, 2, \dots, L \quad (4)$$

where l indicates the distance from the initial entity set, the subscript symbol o^1 is a uniform placeholder for symbol u or v . Given the definition of entity set, we then define the l -th triple set for user u and item v as follows:

$$\mathcal{S}_o^l = \{(h, r, t) | (h, r, t) \in \mathcal{G} \text{ and } h \in \mathcal{E}_o^{l-1}\}, \quad l = 1, 2, \dots, L \quad (5)$$

Taking knowledge graph as side information to construct our model makes great sense, since neighboring entities can be regarded as a intuitively extension of user's preferences and item's features. As illustrated in Figure 2, the initial entity set obtained through collaborative propagation, which is analogous to the source of sound waves, spreads in the medium of KG layer by layer, from near to distant. Through knowledge-based deep propagation, the knowledge-based high-order interaction information of user and item is successfully captured, which effectively improves the model's ability to represent user and item with latent vectors.

¹This paper uses symbol o as a uniform placeholder for user symbol u and item symbol v , since the calculation and formulation for user and item in this article are similar in many scenarios.

3.2 Knowledge-aware Attentive Embedding

Each tail entity has different meanings and latent vector representations when it has diverse head entities and relations in KG. For example, *Forrest Gump* and *Cast Away* have more in common in terms of their directors or casts, but are less similar if measured by genres or writers. As a result, we propose a knowledge-aware attentive embedding method to generate different attentive weights of the tail entity to reveal the different meanings it has when it gets different head entities and relations.

Consider (h, r, t) is the i -th triple in the l -th layer triple set, we build a tail entity's attentive embedding a_i as such:

$$a_i = \pi(e_i^h, r_i) e_i^t \quad (6)$$

where e_i^h is the head entity's embedding, r_i is the relation's embedding and e_i^t is the tail entity's embedding of the i -th triple. $\pi(e_i^h, r_i)$ controls the attentive weight generated from head entity and the relationship between head and tail. We implement the function $\pi(\cdot)$ via a neural network similar to attention mechanism [18], which is formulated as follows:

$$\begin{aligned} z_0 &= \text{ReLU}(W_0(e_i^h || r_i) + b_0) \\ \pi(e_i^h, r_i) &= \sigma(W_2 \text{ReLU}(W_1 z_0 + b_1) + b_2) \end{aligned} \quad (7)$$

where we select ReLU [3] as the nonlinear activation function, and the last activation function is Sigmoid [5]. $||$ is the concatenation operation. W and b are the trainable weight matrices and biases, and their different subscripts indicate that they are parameters of different layers. Hereafter, we normalize the coefficients across the whole triples in the triple set by adopting the softmax function [12]:

$$\pi(e_i^h, r_i) = \frac{\exp(\pi(e_i^h, r_i))}{\sum_{(h', r', t') \in \mathcal{S}_o^l} \exp(\pi(e_{h'}^h, r_{t'}'))} \quad (8)$$

where $\mathcal{S}_o^l$ is the l -th layer triple set for user or item. As a result, the attention weight is capable of suggesting which neighboring tail entity should be given more attention to capture knowledge associations more effectively.

Finally, we obtain representation of the l -th layer triple set for user u or item v :

$$e_o^{(l)} = \sum_{i=1}^{|\mathcal{S}_o^l|} a_i^{(o)}, \quad l = 1, 2, \dots, L \quad (9)$$

where the subscript and superscript o is a uniform placeholder for symbol u or v , and $|\mathcal{S}_o^l|$ is the number of triples in set $\mathcal{S}_o^l$.

Notice that, because the entities in initial entity set which are like the seed for knowledge-based propagation are closest to the original representation, the initial entity set has strong connections with original user and item. Accordingly, we add representations of the initial entity set for both user and item:

$$e_o^{(0)} = \frac{\sum_{e \in \mathcal{E}_o^0} e}{|\mathcal{E}_o^0|} \quad (10)$$

Specially, item v has relevant entities for its original representation while user u does not. The original relevant entities are the closest nodes to item v itself in latent semantic space. As a consequence,

we add it to the representation set of the item v and define it as follows:

$$e_v^{(origin)} = \frac{\sum_{e \in \{e|(e,v) \in \mathcal{A}\}} e}{|\{e|(e,v) \in \mathcal{A}\}|} \quad (11)$$

After knowledge-aware attentive embedding, we formulate the representation set which contains both knowledge-based attentive weighted representations and additional representations for user u and item v as follows:

$$\begin{aligned} \mathcal{T}_u &= \{e_u^{(0)}, e_u^{(1)}, \dots, e_u^{(L)}\} \\ \mathcal{T}_v &= \{e_v^{(origin)}, e_v^{(0)}, e_v^{(1)}, \dots, e_v^{(L)}\} \end{aligned} \quad (12)$$

3.3 Model Prediction

The representation in each layer can be interpreted as hierarchical potential influence which emphasizes different high-order connectivity and preference similarity. We implement three types of aggregators to aggregate the multiple representations in Equations (12) into a single vector for both user and item.

3.3.1 Sum aggregator. Sum aggregator takes the summation of multiple vectors in the set of representations before applying non-linear transformation:

$$agg_{sum}^{(o)} = \sigma \left(W_a \cdot \sum_{e_o \in \mathcal{T}_o} e_o + b_a \right) \quad (13)$$

where we set nonlinear function as Sigmoid [5], and W_a and b_a are the trainable weight and bias. We use the same symbols and nonlinear function for the other two aggregators.

3.3.2 Pooling aggregator. Pooling aggregator takes the maximum value of multiple vectors as the same latitude, followed by a non-linear transformation:

$$agg_{pool}^{(o)} = \sigma \left(W_a \cdot pool_{max}(\mathcal{T}_o) + b_a \right) \quad (14)$$

3.3.3 Concat aggregator. Concat aggregator [4] concatenates representation vectors in the representation set and uses a nonlinear transformation as follows:

$$agg_{concat}^{(o)} = \sigma \left(W_a \cdot (e_o^{(i_1)} || e_o^{(i_2)} || \dots || e_o^{(i_n)}) + b_a \right) \quad (15)$$

where $e_o^{(i_k)} \in \mathcal{T}_o$, and $||$ is the concatenation operation.

Notice that, since the aggregated vector representations of user and item need to have same dimensions for the next calculation (e.g. inner product), we select a balanced combination of multiple representations for the three aggregators. The detail we will discuss in Section 4.5.2.

We use e_u to represent the aggregated vector of user. Analogously, as to item, e_v is declared. Finally, we conduct inner product of representations to predict the user's preference score for the item:

$$\hat{y}_{uv} = e_u^T e_v \quad (16)$$

3.4 Loss Function

To balance the number of positive and negative samples and to guarantee the effect of model training, we extract the same number of negative samples as that of positive samples for each user. The

Table 1: Statistics of the four datasets: Last.FM (Music), Book-Crossing (Book), Movie-Lens20M (Movie) and Dianping-Food (Restaurant). The inter-avg means the average inter-actions per user, the link-avg means the average links per item.

	Music	Book	Movie	Restaurant
# users	1,872	17,860	138,159	2,298,698
# items	3,846	14,967	16,954	1,362
# interactions	42,346	139,746	13,501,622	23,416,418
# inter-avg	23	8	98	10
# entities	9,366	77,903	102,569	28,115
# relations	60	25	32	7
# KG triples	15,518	15,1500	499,474	160,519
# link-avg	4	10	29	118

details of negative sampling are in Section 4.1. Finally, we have the following loss function for CKAN:

$$\mathcal{L} = \sum_{u \in \mathcal{U}} \left(\sum_{v \in \{v|(u,v) \in \mathcal{P}^+\}} \mathcal{J}(y_{uv}, \hat{y}_{uv}) - \sum_{v \in \{v|(u,v) \in \mathcal{P}^-\}} \mathcal{J}(y_{uv}, \hat{y}_{uv}) \right) + \lambda ||\Theta||_2^2 \quad (17)$$

where $\mathcal{J}$ is the cross-entropy loss, $\mathcal{P}^+$ means positive user-item pair set while $\mathcal{P}^-$ is the opposite. $\Theta = \{E, R, W_i, b_i, \forall i \in \{0, 1, 2, a\}\}$ is the set of model parameters, E and R are the embedding tables for all entities and relations, and $||\Theta||_2^2$ is the $L2$ -regularizer that parameterized by λ .

4 EXPERIMENTS

In this section, we evaluate the proposed model under four real-world scenarios, with the aim of answering the following research questions:

- **RQ1:** How does CKAN perform compared with state-of-the-art KG-based recommendation methods?
- **RQ2:** How do different hyper-parameter settings (e.g. depth of the knowledge-aware propagation layer, attention mechanism, aggregation selection) affect CKAN?

4.1 Dataset Description

To evaluate the effectiveness and universality of CKAN, we conduct experiments under different scenarios: music, book, movie, and restaurant recommendations. In addition to domains, the four respective datasets are also varied in size and sparsity. Among these datasets, the first three are publicly accessible and the last one is contributed by Meituan-Dianping Group[23].

- **Last.FM**² where tracks are identified as items is provided by last.fm online music system.
- **Book-Crossing**³ is collected from the book-crossing community, which consists of trenchant ratings (ranging from 0 to 10) from different readers about various books.

²<https://grouplens.org/datasets/hetrec-2011/>

³<http://www2.informatik.uni-freiburg.de/cziegler/BX/>

- **MovieLens-20M**⁴ that contains nearly 20 million explicit ratings (ranging from 1 to 5) of 27 thousand movies by 138 thousand users on the MovieLens website is widely used in movie recommendations.
- **Dianping-Food** that is comprised of more than 10 million interactions (including clicks, purchases, and adding to favorites) between around 2 million users and 1 thousand restaurants is from Dianping.com⁵.

Since the interactions in Last.FM, Book-Crossing and MovieLens-20M are explicit feedback, they need to be transformed into the implicit one where 1 indicates the positive samples (the threshold of the rating to be viewed as positive is 4 for MovieLens-20M, while no threshold is set for Last.FM and Book-Crossing due to their sparsity). As to negative samples, for each user, we randomly sample from his unwatched items with the size equal to his positive ones.

In addition to the interactions between users and items, the sub-KGs for each dataset need to be constructed. For MovieLens-20M, Book-Crossing, and Last.FM datasets, we follow the work in [20, 24] by constructing sub-KGs from the KG called Satori⁶ from Microsoft. For Dianping-Food dataset, we use the extracted sub-KG contributed by [24] from Meituan-dianping Group. Each sub-KG, the format of which is triples, is a subset of the whole KG with a confidence level greater than 0.9. For the sake of simplicity, we exclude the items matched with multiple entities and those unmatched to any of entities. Table 1 summarizes the detailed statistics of the four datasets: Last.FM (music), Book-Crossing (book), MovieLens-20M (movie), and Dianping-Food (restaurant).

4.2 Baselines

To demonstrate the effectiveness of our proposed CKAN, we compare CKAN with four types of recommendation methods: CF-based method (BPRMF), embedding-based method (CKE), path-based method (PER) and propagation-based methods (RippleNet, KGCN, KGNN-LS, KGAT), as follows:

- **BPRMF** [13] is a classical CF method which uses pairwise matrix factorization for implicit feedback optimized by the Bayesian personalized ranking loss.
- **CKE** [33] is a typical embedding-based model combining CF module with structural, textual, and visual knowledge embeddings of items in a unified Bayesian framework. We implement CKE in a simplified way where we use KG embeddings derived from TransR [11] to enhance matrix factorization.
- **PER** [32] is a representative path-based method which treats the KG as heterogeneous information networks and extracts meta-path based latent features to represent the connectivity between users and items along different types of relation paths.
- **RippleNet** [20] is a state-of-the-art propagation-based model which uses a memory-like network and propagates users' potential preferences in the KG to enrich user representations.
- **KGCN** [24] is another state-of-the-art propagation-based model which extends non-spectral GCN approaches to the knowledge graph by aggregating neighborhood information selectively and biasedly, which is able to learn both structure information and

semantic information of the KG as well as users' personalized and potential interests.

- **KGNN-LS** [23] is another state-of-the-art propagation-based model which transforms heterogeneous KG into a user-specific weighted graph and computes personalized item embeddings in graph neural network with label smoothness regularization.
- **KGAT** [26] is also the state-of-the-art propagation-based model which combines the UIG and KG together as a homogeneous unified graph called CKG. Compared with KGCN, KGAT utilizes an attention mechanism to discriminate the importance of neighbors in CKG during propagation. In our implementation, we use the pre-trained embeddings of users and items from BPRMF model as the initialized embeddings.

4.3 Experimental Settings

We divide each dataset into training, evaluation, and test sets with the proportion of 6:2:2. In our experiments, we consider two recommendation scenarios. (1) In click-through rate (CTR) prediction, we use the model learnt from the training set to predict the probability of each interaction in the test set. (2) In top- K recommendation, we apply the trained model to select K items with highest predicted probability for each user in the test set. As to the metrics of all methods, we choose *AUC* and *F1* to evaluate the model for CTR prediction and *Recall@K* for top- K recommendation. In terms of optimization, adam [9] is used to optimize all models, where the batch size is fixed as 1024. To initialize the model parameters, we adopt the default Xavier initializer [2].

We implement our CKAN model in PyTorch⁷ and apply a grid search for hyper-parameters. The learning rate is tuned among $\{10^{-3}, 5 \times 10^{-3}, 10^{-2}, 5 \times 10^{-2}\}$, the coefficient of $L2$ normalization is tuned amongst $\{10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}\}$, and the embedding size is searched in $\{8, 16, 32, 64, 128, 256\}$. Since the optimal size of triple set for user and item might be different, we search the set size in $\{4, 8, 16, 32, 64\}$ for user and item respectively. In addition, we empirically find out the optimal settings for attention network as follows: the number of layers is set to 2 and the dimension of hidden layer is set as the same dimension of the embedding size. The best settings for hyper-parameters in all comparison methods are reached by either empirical study or following the original papers.

4.4 Performance Comparison (RQ1)

Table 2 summarizes the results of CTR prediction on four datasets, and the curves of top- K recommendation on four datasets are shown respectively in Figure 3. Intuitively, CKAN has made great improvements over state-of-the-art baselines in both recommendation scenarios. We have the following observations from the experimental results:

- Compared with these state-of-the-art baselines, CKAN achieves strongly competitive performance on all four datasets. Specifically, CKAN improves over the state-of-the-art baselines *w.r.t.* *AUC* by 1.6%, 2.9% and 1.7% in Last.FM, Book-Crossing and Dianping-Food datasets, respectively. Notice that, CKAN does

⁴<https://grouplens.org/datasets/movielens/>

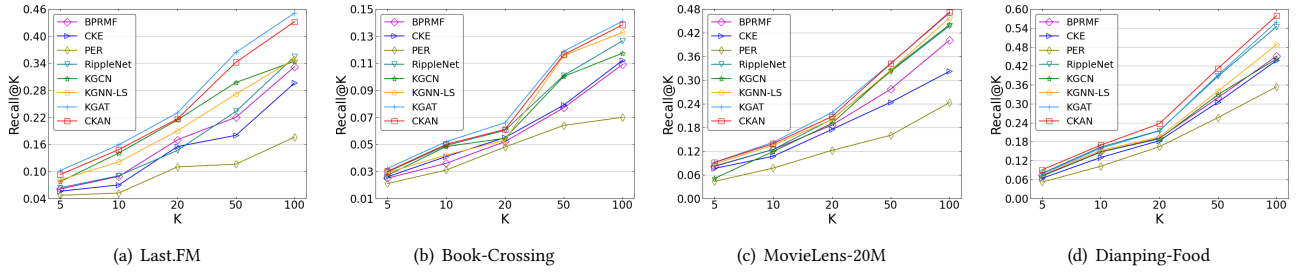
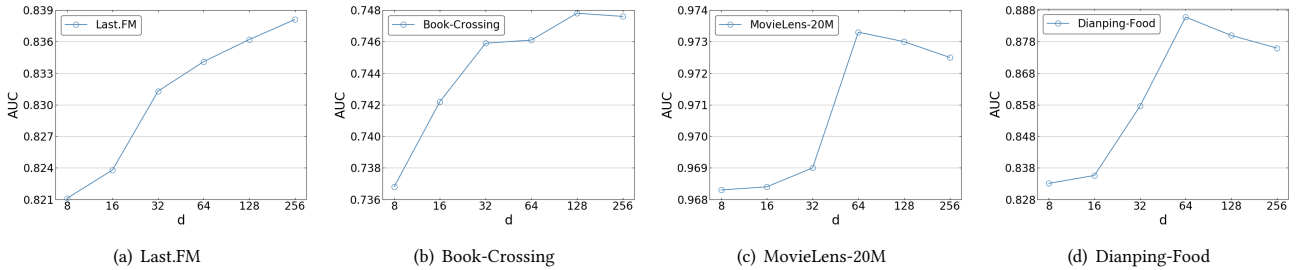
⁵<https://www.dianping.com/>

⁶<https://searchengineland.com/library/bing/bing-satori>

⁷<https://pytorch.org>

Table 2: The result of AUC and $F1$ in CTR prediction. The imp-avg means the average improvements across all baselines.

model	Last.FM		Book-Crossing		MovieLens-20M		Dianping-Food	
	AUC	$F1$	AUC	$F1$	AUC	$F1$	AUC	$F1$
BPRMF	0.756 (-10.2%)	0.701 (-8.8%)	0.658 (-12.6%)	0.611 (-9.2%)	0.958 (-1.8%)	0.914 (-1.6%)	0.832 (-5.2%)	0.764 (-4.7%)
CKE	0.747 (-11.3%)	0.674 (-12.4%)	0.676 (-10.2%)	0.623 (-7.4%)	0.927 (-5.0%)	0.874 (-5.9%)	0.812 (-7.5%)	0.741 (-7.6%)
PER	0.641 (-23.9%)	0.603 (-21.6%)	0.605 (-19.7%)	0.572 (-15.0%)	0.838 (-14.1%)	0.792 (-14.7%)	0.766 (-12.8%)	0.697 (-13.1%)
RippleNet	0.776 (-7.8%)	0.702 (-8.7%)	0.721 (-4.2%)	0.647 (-3.9%)	0.976 (0.0%)	0.927 (-0.2%)	0.863 (-1.7%)	0.783 (-2.4%)
KGCN	0.802 (-4.8%)	0.708 (-7.9%)	0.684 (-9.2%)	0.631 (-6.2%)	0.977 (+0.1%)	0.930 (+0.1%)	0.845 (-3.8%)	0.774 (-3.5%)
KGNN-LS	0.805 (-4.4%)	0.722 (-6.1%)	0.676 (-10.2%)	0.631 (-6.2%)	0.975 (-0.1%)	0.929 (0.0%)	0.852 (-3.0%)	0.778 (-3.0%)
KGAT	0.829 (-1.6%)	0.742 (-3.6%)	0.731 (-2.9%)	0.654 (-2.8%)	0.976 (0.0%)	0.928 (-0.1%)	0.846 (-3.6%)	0.785 (-2.1%)
CKAN	0.842	0.769	0.753	0.673	0.976	0.929	0.878	0.802

**Figure 3: The result of $Recall@K$ in top- K recommendation.****Figure 4: The result of AUC w.r.t. different dimensions on all four datasets.**

not perform best on MovieLens-20M, since the average interactions per user are so abundant that the propagation of knowledge is almost ineffective.

- In top- K recommendation, the performance of CKAN is excellent especially when K becomes large in MovieLens-20M and Dianping-Food datasets. Compared with RippleNet, the performance of CKAN justifies the effectiveness of the knowledge-aware attention mechanism. The results compared with KGCN and KGNN-LS show the significance of explicitly encoding collaborative signals.
- Notice that KGAT have a more prominent result *w.r.t.* $Recall$ in Last.FM and Book-Crossing datasets, but its performance is surpassed by CKAN in MovieLens-20M and Dianping-Food especially when K is larger. It can be inferred that KGAT is capable of giving higher scores to the top- K positive samples in sparse and small-scale datasets. However, when the datasets becomes

denser and larger, more noise will be introduced to KGAT during propagation in CKG. Therefore, the combination of collaborative signals and knowledge associations in CKAN can be more effective when K is larger in a large dataset.

- The ranking of the results of all models' performance on the four datasets from high to low is movie, restaurant, music, book. One possible explanation is that the average interactions per user and the average links per item are different on four datasets. For instance, the movie and restaurant datasets have richer interactions and links on average than the music and book datasets. As a result, there is not enough information on the datasets which have poorer interactions and links for recommender models to learn the latent embeddings.
- Judging from the experimental results, it is obvious that the CF-based methods are overwhelmed by most KG-based methods, especially the propagation-based methods on the four datasets

across all the evaluation metrics. This experimental results demonstrate that the usage of KG is of great help for recommendation. It is worth noting that the performance of BPRMF is better than CKE on some indicators, which illustrates that modeling the first-order relationship in KG might not fully utilize the rich information encoded in KG. From another aspect, this phenomenon indicates the effectiveness of modeling high-order knowledge information.

- Through the comparison of results of all KG-based methods, the propagation-based methods are superior to the embedding-based methods and the path-based methods on all four datasets. This verifies the significance of propagating knowledge-based associations and modeling the high-order connectivity of neighbors in KG. In all KG-based methods, the performance of the path-based method (PER) is worse than others because it is actually difficult to define optimal meta-paths. The embedding-based approach (CKE) does not perform well, which might be caused by a lack of visual and textual data.

4.5 Study of CKAN (RQ2)

4.5.1 Effect of Dimension of Embedding. We use the same dimensional parameters for the embeddings of entity and relation to facilitate calculations and explore the impact of different dimensions on the performance of CKAN on all four datasets. The results shown in Figure 4 reveal the following findings: Intuitively, increasing the dimension d of embeddings within a certain range can effectively improve the performance of CKAN, but the performance will decrease if d is too large. A reasonable explanation is that more information can be encoded when d becomes larger, but it may also cause a little overfitting when d is larger than the threshold. It's worth noticing that CKAN has stronger tolerance for dimension selection compared with other baselines, which means selecting dimensions within a certain range has little effect on results. This feature reduces the dependence of experimental results on hyper-parameters, making experimental results easy to reproduce.

4.5.2 Effect of Depths of Layer. We vary the maximal depths of layer L of knowledge-aware propagation to investigate how performance changes in CKAN. It should be noted that *concat* aggregator needs to ensure the same number of representations for calculation, but the number of representations in $\mathcal{T}_u$ and $\mathcal{T}_v$ is different. Therefore, we remove $e_v^{(0)}$ in $\mathcal{T}_v$ as a compromise solution to keep the quantity consistent. For instance, $L = 0$ means $e_u^{(0)}$ and $e_v^{(origin)}$ to aggregate, and so on.

The experimental results of different depths of layer are presented in Table 3, from which we can observe that: The best performance is achieved when L is 3, 2, 1 and 1 in music, book, movie and restaurant, respectively. One possible reason for this phenomenon is that long-distance propagation is like a double-edged sword, which provides more supplementary knowledge information and also brings more noise especially when the amount of data is large. Maintaining a reasonable depth of propagation layers can maximize the use of knowledge information in different scenarios.

4.5.3 Effect of Aggregators. To explore the impact of aggregators, we experiment with CKAN under different settings of aggregator,

Table 3: The result of AUC w.r.t. different depths of layer.

Depth of Layer L	0	1	2	3
music	0.829	0.844	0.841	0.845
book	0.724	0.746	0.753	0.747
movie	0.961	0.973	0.972	0.971
restaurant	0.868	0.877	0.876	0.873

Table 4: The result of AUC w.r.t. different aggregators.

Aggregator	agg_{sum}	agg_{pool}	agg_{concat}
music	0.821	0.823	0.839
book	0.716	0.732	0.748
movie	0.964	0.958	0.972
restaurant	0.872	0.870	0.879

Table 5: The result of AUC w.r.t. effect of collaborative propagation and knowledge-aware attention mechanism.

Category	CKAN _{deep CP}	CKAN _{w/o Att}	CKAN
music	0.823	0.801	0.842
book	0.742	0.737	0.750
movie	0.963	0.965	0.975
restaurant	0.857	0.871	0.880

more specifically, *Sum*, *Pool* and *Concat* aggregator (cf. Section 3.3.3). Since agg_{sum} and agg_{pool} do not need to guarantee the same number of representations used, we add additional $e_v^{(0)}$ for them to aggregate to retain more information. The experimental results are recorded in Table 4. We have the following observation by investigating the experimental results: agg_{concat} is consistently superior to agg_{sum} and agg_{pool} . This may be because the *Concat* aggregator is capable of retaining more information hidden in embeddings compared with the other two aggregators.

4.5.4 Effect of Collaborative Propagation and Knowledge-aware Attention Mechanism. To verify the impact of collaborative propagation and attention mechanism, we study two variants of CKAN. For collaborative propagation, we use a variant with two-layer collaborative propagation to verify the effectiveness of using first-order collaborative information, which is termed as CKAN_{deepCP}. For attention mechanism, we disable the knowledge-aware attention mechanism (cf. Equation 6) and set $\pi(e_i^h, r_i)$ as $1/|S_o^i|$, termed CKAN_{w/o Att}. In the analyses of data from Table 5, we have the following findings:

- The variant with two-layer collaborative propagation performs worse than the base CKAN which has first-order collaborative propagation on the four datasets. This phenomenon proves that deeper collaborative propagation might introduce more noise in the representations as a result of the instability and uncertainty of the interactions. The nearest interacted items could represent preferences of users more effectively.

Table 6: The result of AUC on Last.FM w.r.t. different sizes of triple set.

item \ user	4	8	16	32	64
4	0.822	0.832	0.834	0.832	0.833
8	0.831	0.834	0.840	0.840	0.841
16	0.820	0.830	0.838	0.837	0.837
32	0.826	0.835	0.838	0.835	0.834
64	0.819	0.830	0.836	0.836	0.838

Table 7: The result of AUC on Book-Crossing w.r.t. different sizes of triple set.

item \ user	4	8	16	32	64
4	0.727	0.730	0.740	0.742	0.736
8	0.733	0.733	0.740	0.741	0.742
16	0.741	0.745	0.749	0.750	0.751
32	0.741	0.745	0.749	0.749	0.750
64	0.743	0.748	0.748	0.749	0.740

- The experimental results of $CKAN_{w/o Att}$ are worse than those of CKAN on all datasets. This supports that knowledge-aware attention mechanism is a powerful determinant, enabling knowledge associations to be more fully encoded into latent vector representations.

4.5.5 Effect of Size of Triple Set. We vary the size of triple set of both user and item to explore the cap of CKAN. The results of AUC on the first two datasets (Last.FM and Book-Crossing) are presented in Table 6 and Table 7, respectively. We summarize the results and find that: Increasing the size of item's triple set can boost the performance. While the results drop if the size of user's triple set is too large. The best performance is obtained when the size of item's triple set is taken as 64 and the size of user's triple set is taken as 8 or 16. One reasonable explanation is that there is a large gap between the number of entities in initial entity sets of user and item. The number of entities determines the amount of triple that can be associated and greatly affects the final performance.

5 CONCLUSION AND FUTURE WORK

In this paper, we propose a novel collaborative knowledge-aware attentive network which is abbreviated to CKAN for recommender systems. CKAN explicitly encodes the collaborative signals that are latent in user-item interactions and naturally combines them with knowledge associations in an end-to-end manner. Specifically, CKAN employs a heterogeneous propagation strategy to encode both kinds of information and utilizes a knowledge-aware attention mechanism to discriminate the contribution of different knowledge-based neighbors. The extensive experiments we conduct on four datasets in the real world demonstrates the superiority of CKAN over other counterparts.

As to future work, there are two feasible avenues: (1) Since this paper focuses on modeling the latent vector representations via

heterogeneous propagation, one direction of further study is to design a better way to aggregate the multiple representations to a single vector which is capable of making full use of the multi-layer representations. (2) Such a combination with knowledge graph provides a new idea to many other structural information existing in the real world such as social networks. It is worth trying to improve the recommendation performance.

ACKNOWLEDGMENTS

The research was supported by the National Key R&D Program of China under Grant No.2018YFB1402600.

REFERENCES

- [1] Yixin Cao, Xiang Wang, Xiangnan He, Zikun Hu, and Tat-Seng Chua. 2019. Unifying Knowledge Graph Learning and Recommendation: Towards a Better Understanding of User Preferences. In *the 28th International Conference on World Wide Web*. ACM, 151–161.
- [2] Xavier Glorot and Yoshua Bengio. 2010. Understanding the difficulty of training deep feedforward neural networks. In *the 13th International Conference on Artificial Intelligence and Statistics*. 249–256.
- [3] Richard HR Hahnloser, Rahul Sarpeshkar, Misha A Mahowald, Rodney J Douglas, and H Sebastian Seung. 2000. Digital selection and analogue amplification coexist in a cortex-inspired silicon circuit. *Nature* 405, 6789 (2000), 947.
- [4] Will Hamilton, Zhitaoying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. In *the 31st Conference on Advances in Neural Information Processing Systems*. 1024–1034.
- [5] Jun Han and Claudio Moraga. 1995. The influence of the sigmoid function parameters on the speed of backpropagation learning. In *International Workshop on Artificial Neural Networks*. Springer, 195–201.
- [6] Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. 2017. Neural collaborative filtering. In *the 26th International Conference on World Wide Web*. ACM, 173–182.
- [7] Binbin Hu, Chuan Shi, Wayne Xin Zhao, and Philip S. Yu. 2018. Leveraging Meta-path Based Context for Top- N Recommendation with A Neural Co-Attention Model. In *the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. ACM, 1531–1540.
- [8] Guoliang Ji, Shizhu He, Liheng Xu, Kang Liu, and Jun Zhao. 2015. Knowledge graph embedding via dynamic mapping matrix. In *the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing*. 687–696.
- [9] Diederik P Kingma and Jimmy Ba. 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980* (2014).
- [10] Yehuda Koren, Robert Bell, and Chris Volinsky. 2009. Matrix factorization techniques for recommender systems. *Computer* 8 (2009), 30–37.
- [11] Yankai Lin, Zhiyuan Liu, Maosong Sun, Yang Liu, and Xuan Zhu. 2015. Learning entity and relation embeddings for knowledge graph completion. In *the 29th AAAI Conference on Artificial Intelligence*.
- [12] Roland Memisevic, Christopher Zach, Marc Pollefeys, and Geoffrey E Hinton. 2010. Gated softmax classification. In *the 24th Conference on Advances in Neural Information Processing Systems*. 1603–1611.
- [13] Steffen Rendle, Christoph Freudenthaler, Zeno Gantner, and Lars Schmidt-Thieme. 2009. BPR: Bayesian personalized ranking from implicit feedback. In *the 25th Conference on Uncertainty in Artificial Intelligence*. AUAI Press, 452–461.
- [14] Xiao Sha, Zhu Sun, and Jie Zhang. 2019. Attentive Knowledge Graph Embedding for Personalized Recommendation. *arXiv preprint arXiv:1910.08288* (2019).
- [15] Chuan Shi, Binbin Hu, Wayne Xin Zhao, and S Yu Philip. 2018. Heterogeneous information network embedding for recommendation. *IEEE Transactions on Knowledge and Data Engineering* 31, 2 (2018), 357–370.
- [16] Zhu Sun, Jie Yang, Jie Zhang, Alessandro Bozzon, Long-Kai Huang, and Chi Xu. 2018. Recurrent knowledge graph embedding for effective recommendation. In *the 12th ACM Conference on Recommender Systems*. ACM, 297–305.
- [17] Xiaoli Tang, Tengyun Wang, Haizhi Yang, and Hengjie Song. 2019. AKUPM: Attention-enhanced knowledge-aware user preference model for recommendation. In *the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. ACM, 1891–1899.
- [18] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *the 31st Conference on Advances in Neural Information Processing Systems*. 5998–6008.
- [19] V Vijayakumar, Subramaniaswamy Vairavasundaram, R Logesh, and A Sivapathi. 2019. Effective knowledge based recommender system for tailored multiple point of interest recommendation. *International Journal of Web Portals* 11, 1 (2019), 1–18.

- [20] Hongwei Wang, Fuzheng Zhang, Jialin Wang, Miao Zhao, Wenjie Li, Xing Xie, and Minyi Guo. 2018. Ripplenet: Propagating user preferences on the knowledge graph for recommender systems. In *the 27th ACM SIGKDD International Conference on Information and Knowledge Management*. ACM, 417–426.
- [21] Hongwei Wang, Fuzheng Zhang, Xing Xie, and Minyi Guo. 2018. DKN: Deep knowledge-aware network for news recommendation. In *the 27th International Conference on World Wide Web*. ACM, 1835–1844.
- [22] Hongwei Wang, Fuzheng Zhang, Miao Zhao, Wenjie Li, Xing Xie, and Minyi Guo. 2019. Multi-Task Feature Learning for Knowledge Graph Enhanced Recommendation. In *the 28th International Conference on World Wide Web*. ACM, 2000–2010.
- [23] Hongwei Wang, Fuzheng Zhang, Mengdi Zhang, Leskovec Jure, Miao Zhao, Wenjie Li, and Zhongyuan Wang. 2019. Knowledge-aware graph neural networks with label smoothness regularization for recommender systems. In *the 25th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*. ACM, 968–977.
- [24] Hongwei Wang, Miao Zhao, Xing Xie, Wenjie Li, and Minyi Guo. 2019. Knowledge graph convolutional networks for recommender systems. In *the 28th International Conference on World Wide Web*. ACM, 3307–3313.
- [25] Quan Wang, Zhendong Mao, Bin Wang, and Li Guo. 2017. Knowledge graph embedding: A survey of approaches and applications. *IEEE Transactions on Knowledge and Data Engineering* 29, 12 (2017), 2724–2743.
- [26] Xiang Wang, Xiangnan He, Yixin Cao, Meng Liu, and Tat-Seng Chua. 2019. KGAT: Knowledge Graph Attention Network for Recommendation. In *the 25th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*. ACM, 950–958.
- [27] Xiang Wang, Dingxian Wang, Canran Xu, Xiangnan He, Yixin Cao, and Tat-Seng Chua. 2019. Explainable reasoning over knowledge graphs for recommendation. In *the AAAI Conference on Artificial Intelligence*, Vol. 33. 5329–5336.
- [28] Zhen Wang, Jianwen Zhang, Jianlin Feng, and Zheng Chen. 2014. Knowledge Graph Embedding by Translating on Hyperplanes.
- [29] Markus Weimer, Alexandros Karatzoglou, and Alex Smola. 2008. Adaptive Collaborative Filtering. In *the 2nd ACM Conference on Recommender Systems*. ACM, 275–282.
- [30] Rex Ying, Ruining He, Kaifeng Chen, Pong Eksombatchai, William L Hamilton, and Jure Leskovec. 2018. Graph convolutional neural networks for web-scale recommender systems. In *the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. ACM, 974–983.
- [31] Xiao Yu, Xiang Ren, Quanquan Gu, Yizhou Sun, and Jiawei Han. 2013. Collaborative filtering with entity similarity regularization in heterogeneous information networks. In *the 2nd International Joint Conference on Artificial Intelligence Workshop on Heterogeneous Information Network Analysis*. Citeseer.
- [32] Xiao Yu, Xiang Ren, Yizhou Sun, Quanquan Gu, Bradley Sturt, Urvashi Khandelwal, Brandon Norick, and Jiawei Han. 2014. Personalized entity recommendation: A heterogeneous information network approach. In *the 7th ACM International Conference on Web Search and Data Mining*. ACM, 283–292.
- [33] Fuzheng Zhang, Nicholas Jing Yuan, Defu Lian, Xing Xie, and Wei-Ying Ma. 2016. Collaborative knowledge base embedding for recommender systems. In *the 22nd ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. ACM, 353–362.
- [34] Hanwang Zhang, Fumin Shen, Wei Liu, Xiangnan He, Huanbo Luan, and Tat-Seng Chua. 2016. Discrete collaborative filtering. In *the 39th International ACM SIGIR conference on Research and Development in Information Retrieval*. ACM, 325–334.
- [35] Ming Zhu, De-sheng Zhen, Ran Tao, You-qun Shi, Xiang-yang Feng, and Qian Wang. 2019. Top-N Collaborative Filtering Recommendation Algorithm Based on Knowledge Graph Embedding. In *the 14th International Conference on Knowledge Management in Organizations*. Springer, 122–134.